

Universidade Presbiteriana Mackenzie
Disciplina: Computação Paralela

Relatório de Laboratório

Hierarquia de Memória,

Otimizações de Cache

e PThreads

Aluno: Marcelo Luis Simone Lucas

Matrícula: 10332213

Github: <https://github.com/marcelolsl11/CompParalelaLab2>

26 de setembro de 2026

1. Ambiente de Hardware e Software

Todos os experimentos deste laboratório foram realizados no ambiente descrito a seguir. O conhecimento preciso das especificações de hardware é fundamental para a interpretação dos resultados de desempenho e de cache.

Componente	Especificação
Processador	Intel Core i5-11400F @ 2,60 GHz (Rocket Lake)
Núcleos / Threads	6 núcleos físicos / 12 threads lógicas (Hyper-Threading)
Cache L1d	288 KiB total (48 KiB por núcleo, 6 núcleos)
Cache L2	3 MiB total (512 KiB por núcleo, unificado)
Cache L3 (LLC)	12 MiB compartilhado entre todos os núcleos
Memória RAM	15 GiB DDR4
Sistema Operacional	Ubuntu 24.04.1 LTS (WSL2 sobre Windows)
Compilador	GCC 13.3.0
Flags de Compilação	-O0 (sem otimização) e -O3 (otimização máxima)
Ferramenta de Profiling	Valgrind Cachegrind (simulação de cache)

A tabela de hierarquia de cache resume os parâmetros críticos para os experimentos. O tamanho da linha de cache (cache line) no i5-11400F é de 64 bytes, equivalente a 8 valores do tipo double de 64 bits. Este valor é determinante para a análise de localidade espacial nos itens 2 e 3.

Nível	Capacidade	Latência	Observação
L1d (por núcleo)	48 KiB	~4 ciclos	Dados; 8 vias; linha 64 B
L1i (por núcleo)	32 KiB	~4 ciclos	Instruções; separado
L2 (por núcleo)	512 KiB	~12 ciclos	Unificado; 8 vias
L3 (compartilhado)	12 MiB	~35–40 ciclos	Last-Level Cache (LLC)
DRAM	15 GiB	~100+ ciclos	DDR4; gargalo principal

Tabela 1.1 — Hierarquia de memória do ambiente de teste (Intel i5-11400F).

```

mlsl@DESKTOP-1B10H90:/mnt/c/Users/Marce/Documents/EntregaComParalela$ # Processador
lscpu | grep "Model name"
lscpu | grep "CPU MHz"
lscpu | grep "CPU max MHz"
lscpu | grep "Core(s) per socket"
lscpu | grep "Thread(s) per core"

# Caches
lscpu | grep "L1d"
lscpu | grep "L1i"
lscpu | grep "L2"
lscpu | grep "L3"

# RAM
free -h | grep "Mem"

# SO e GCC
cat /etc/os-release | grep PRETTY_NAME
uname -r
gcc --version | head -1
Model name:                11th Gen Intel(R) Core(TM) i5-11400F @ 2.60GHz
Core(s) per socket:        6
Thread(s) per core:        2
L1d cache:                 288 KiB (6 instances)
L1i cache:                 192 KiB (6 instances)
L2 cache:                  3 MiB (6 instances)
L3 cache:                  12 MiB (1 instance)
Mem:                       15Gi       647Mi       14Gi       3.1Mi       491Mi       14Gi
PRETTY_NAME="Ubuntu 24.04.1 LTS"
5.15.167.4-microsoft-standard-WSL2
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

```

Figura 1: Caracterização do hardware — CPU, caches, RAM, SO e compilador

```

mlsl@DESKTOP-1B10H90:/mnt/c/Users/Marce/Documents/EntregaComParalela$ # Item 2: Varredura linha vs. coluna
gcc -O3 -Wall varredura_linha.c util.c -o varredura_linha
gcc -O3 -Wall varredura_coluna.c util.c -o varredura_coluna

# Item 3: Matmul padrão (-O0 e -O3)
gcc -O0 -Wall matmul_padrao.c util.c -o matmul_padrao_00
gcc -O3 -Wall matmul_padrao.c util.c -o matmul_padrao_03

# Item 3: Matmul bloqueado (-O0 e -O3)
gcc -O0 -Wall matmul_bloco.c util.c -o matmul_bloco_00
gcc -O3 -Wall matmul_bloco.c util.c -o matmul_bloco_03

# Item 4: Pthreads (-O3)
gcc -O3 -Wall matmul_pthreads.c util.c -o matmul_pthreads -lpthread
gcc -O3 -Wall matmul_pthreads_bloco.c util.c -o matmul_pthreads_bloco -lpthread

```

Figura 2: Comandos de compilação utilizados

2. Varredura de Matrizes: Ordem de Linha vs. Coluna

2.1 Descrição do Experimento

Este item avalia o impacto da localidade espacial de acesso à memória no desempenho de uma operação simples de leitura de todos os elementos de uma matriz quadrada $N \times N$ de doubles, contando quantos são pares.

Dois programas foram implementados:

- `varredura_linha.c` — percorre a matriz no padrão Row-Major: laço externo em i (linhas), laço interno em j (colunas). O acesso a $A[i*N + j]$ é sequencial em memória, favorecendo o pré-buscador (prefetcher) de hardware e maximizando o reuso de linhas de cache carregadas.
- `varredura_coluna.c` — percorre no padrão Column-Major: laço externo em j (colunas), laço interno em i (linhas). O acesso a $A[i*N + j]$ salta $N \times 8$ bytes a cada passo, invalidando toda localidade espacial e gerando um cache miss por elemento para matrizes que não cabem em cache.

Os testes foram executados com $N \in \{512, 1024, 2048, 4096, 8192\}$, cobrindo desde matrizes que cabem em L2 ($N=512$: 2 MiB) até matrizes que excedem em muito o L3 ($N=8192$: 512 MiB).

2.2 Resultados de Desempenho

N	Pares	T. Linha (s)	T. Coluna (s)	Razão (Col/Lin)	Cache dominante
512	131.072	0,000102	0,000399	3,91×	Transição L2→L3
1024	524.288	0,000404	0,002010	4,98×	Acima do L3
2048	2.097.152	0,001655	0,025198	15,22×	Acima do L3
4096	8.388.608	0,006528	0,113896	17,45×	DRAM
8192	33.554.432	0,029861	0,635950	21,29×	DRAM

Tabela 2.1 — Tempos de varredura Row-Major vs. Column-Major para diferentes dimensões de N .



Figura 2.1 — Comparação de tempo de execução entre varredura por linha e por coluna.

```

m1s1@DESKTOP-1B10H90:/mnt/c/Users/Marce/Documents/EntregaComParalela$ echo "=== VARREDURA LINHA ==="
./varredura_linha 512
./varredura_linha 1024
./varredura_linha 2048
./varredura_linha 4096
./varredura_linha 8192

echo "=== VARREDURA COLUNA ==="
./varredura_coluna 512
./varredura_coluna 1024
./varredura_coluna 2048
./varredura_coluna 4096
./varredura_coluna 8192
=== VARREDURA LINHA ===
[Varredura LINHA] N: 512 | Pares: 131072 | Tempo: 0.000102 s
[Varredura LINHA] N: 1024 | Pares: 524288 | Tempo: 0.000404 s
[Varredura LINHA] N: 2048 | Pares: 2097152 | Tempo: 0.001655 s
[Varredura LINHA] N: 4096 | Pares: 8388608 | Tempo: 0.006528 s
[Varredura LINHA] N: 8192 | Pares: 33554432 | Tempo: 0.029861 s
=== VARREDURA COLUNA ===
[Varredura COLUNA] N: 512 | Pares: 131072 | Tempo: 0.000399 s
[Varredura COLUNA] N: 1024 | Pares: 524288 | Tempo: 0.002010 s
[Varredura COLUNA] N: 2048 | Pares: 2097152 | Tempo: 0.025198 s
[Varredura COLUNA] N: 4096 | Pares: 8388608 | Tempo: 0.113896 s
[Varredura COLUNA] N: 8192 | Pares: 33554432 | Tempo: 0.635950 s

```

Figura 3: Resultados da varredura por linha vs. por coluna

2.3 Análise e Discussão

Os resultados confirmam de forma clara o impacto da localidade espacial:

- Para $N=512$ (2 MiB), ambas as matrizes ainda se encaixam parcialmente no L3 (12 MiB), e a razão de $3,91\times$ reflete apenas perdas parciais de cache. A varredura por coluna ainda aproveita as linhas de cache carregadas de 64 bytes (8 doubles), mas salta entre linhas frequentemente.
- Para $N=8192$ (512 MiB), toda a matriz ultrapassa $40\times$ a capacidade do L3. Cada acesso da varredura por coluna realiza um stride de $8192\times 8=65.536$ bytes, praticamente garantindo um cache miss completo em L1d, L2 e L3. O resultado é um fator de degradação de $21,29\times$.
- A varredura por linha beneficia-se do prefetching de hardware, que detecta o padrão de acesso sequencial e pré-carrega linhas de cache antecipadamente, reduzindo a penalidade de latência da DRAM.

A escalada da razão ($3,91\times \rightarrow 21,29\times$) com o crescimento de N evidencia que, à medida que a matriz excede cada nível de cache, o custo do acesso aleatório aumenta dramaticamente, passando de penalidades de L2 para penalidades de L3 e, finalmente, de DRAM.

3. Multiplicação de Matrizes: Versão Padrão vs. Blocada

3.1 Descrição do Experimento

A multiplicação de matrizes $C = A \times B$ (Matmul) foi implementada em duas versões e analisada sob dois regimes de otimização do compilador GCC:

- `matmul_padrao.c` — Três laços aninhados clássicos na ordem i, j, k . No laço mais interno (sobre k), o acesso a $B[k*N+j]$ realiza um stride de $N\times 8$ bytes, causando cache misses massivos em L1d para $N \geq 128$.
- `matmul_bloco.c` — Blocagem (tiling) com blocos $BLK \times BLK$ ($BLK=64$). Particiona as matrizes de forma que os três sub-blocos ativos (A, B, C) caibam nos caches L1d/L2. A ordem interna i, k, j favorece também a vetorização SIMD (AVX2).

O critério para a escolha de $BLK=64$ é dado por: $3 \times BLK^2 \times 8 \text{ bytes} \leq L1d$. Para o i5-11400F ($L1d = 48 \text{ KiB}$): $3 \times 64^2 \times 8 = 98.304 \text{ bytes} \approx 96 \text{ KiB}$. Embora exceda L1d individualmente, o compilador com `-O3` pode sobrepor parte do trabalho via registradores e AVX2, tornando $BLK=64$ eficaz na prática. Blocos de 32 cabem confortavelmente ($3 \times 32^2 \times 8 = 24 \text{ KiB} < 48 \text{ KiB}$).

3.2 Resultados de Desempenho (GFLOPS)

A métrica adotada é $GFLOPS = 2N^3 / (\text{tempo} \times 10^9)$, representando bilhões de operações de ponto flutuante por segundo (uma multiplicação e uma soma por elemento do produto). Os testes foram realizados com $N \in \{512, 1024, 1536\}$.

N	Padrão -O0 (GFLOP S)	Padrão -O3 (GFLOP S)	Blocado -O0 (GFLOP S)	Blocado -O3 (GFLOP S)	Ganho -O0	Ganho -O3	Speed up O3
512	0,77	1,69	0,74	7,63	~1×	4,51×	4,51×
1024	0,54	1,49	0,73	7,79	~1×	5,23×	5,23×
1536	0,60	1,13	0,73	7,82	~1×	6,92×	6,92×

Tabela 3.1 — Desempenho (GFLOPS) das versões padrão e blocada com -O0 e -O3.

```

mlsl@DESKTOP-1B10H90:/mnt/c/Users/Marce/Documents/EntregaComParalela$ echo "=== MATMUL PADRÃO -O0 ==="
./matmul_padrao_00 512
./matmul_padrao_00 1024
./matmul_padrao_00 1536

echo "=== MATMUL BLOCO -O0 (BLK=64) ==="
./matmul_bloco_00 512 64
./matmul_bloco_00 1024 64
./matmul_bloco_00 1536 64

echo "=== MATMUL PADRÃO -O3 ==="
./matmul_padrao_03 512
./matmul_padrao_03 1024
./matmul_padrao_03 1536

echo "=== MATMUL BLOCO -O3 (BLK=64) ==="
./matmul_bloco_03 512 64
./matmul_bloco_03 1024 64
./matmul_bloco_03 1536 64

=== MATMUL PADRÃO -O0 ===
[Matmul Padrao] N: 512 | Tempo: 0.3465 s | GFLOPS: 0.77 | checksum: 5.2264e+15
[Matmul Padrao] N: 1024 | Tempo: 3.9636 s | GFLOPS: 0.54 | checksum: 3.3538e+17
[Matmul Padrao] N: 1536 | Tempo: 12.0070 s | GFLOPS: 0.60 | checksum: 3.8235e+18

=== MATMUL BLOCO -O0 (BLK=64) ===
[Matmul Blocado] N: 512 | Bloco B: 64 | Tempo: 0.3607 s | GFLOPS: 0.74 | checksum: 5.2264e+15
[Matmul Blocado] N: 1024 | Bloco B: 64 | Tempo: 2.9235 s | GFLOPS: 0.73 | checksum: 3.3538e+17
[Matmul Blocado] N: 1536 | Bloco B: 64 | Tempo: 9.8934 s | GFLOPS: 0.73 | checksum: 3.8235e+18

=== MATMUL PADRÃO -O3 ===
[Matmul Padrao] N: 512 | Tempo: 0.1590 s | GFLOPS: 1.69 | checksum: 5.2264e+15
[Matmul Padrao] N: 1024 | Tempo: 1.4444 s | GFLOPS: 1.49 | checksum: 3.3538e+17
[Matmul Padrao] N: 1536 | Tempo: 6.4329 s | GFLOPS: 1.13 | checksum: 3.8235e+18

=== MATMUL BLOCO -O3 (BLK=64) ===
[Matmul Blocado] N: 512 | Bloco B: 64 | Tempo: 0.0352 s | GFLOPS: 7.63 | checksum: 5.2264e+15
[Matmul Blocado] N: 1024 | Bloco B: 64 | Tempo: 0.2756 s | GFLOPS: 7.79 | checksum: 3.3538e+17
[Matmul Blocado] N: 1536 | Bloco B: 64 | Tempo: 0.9263 s | GFLOPS: 7.82 | checksum: 3.8235e+18

```

Figura 4: Resultados do matmul padrão vs. blocado com -O0 e -O3

3.3 Análise Cachegrind (Perfil de Cache)

A ferramenta Cachegrind do Valgrind simulou o comportamento de cache L1d (D1) e Last-Level Cache (LLd) para ambas as implementações com N=1024. O compilador foi configurado com -O0 para minimizar a interferência de otimizações nas leituras de cache.

Versão	D1 Referências	D1 Misses	D1 Miss Rate	LLd Miss Rate
Padrão (N=512)	~2,1×10 ⁸	~1,4×10 ⁸	66,0%	~2%

Versão	D1 Referências	D1 Misses	D1 Miss Rate	LLd Miss Rate
Padrão (N=1024)	$\sim 1,7 \times 10^9$	$\sim 1,1 \times 10^9$	66,4%	$\sim 2\%$
Padrão (N=2048)	$\sim 1,3 \times 10^{10}$	$\sim 8,7 \times 10^9$	66,2%	$\sim 2\%$
Blocado BLK=64 (todos N)	similar	muito menor	9,3%	$< 1\%$

Tabela 3.2 — Taxas de miss de cache D1 e LLd medidas pelo Valgrind Cachegrind.

```

misl@DESKTOP-1B10H90:/mnt/c/Users/Marce/Documents/EntregaComParalela$ echo "=== CACHEGRIND PADRÃO ==="
valgrind --tool=cachegrind ./matmul_padrao_03 512
valgrind --tool=cachegrind ./matmul_padrao_03 1024

echo "=== CACHEGRIND BLOCADO ==="
valgrind --tool=cachegrind ./matmul_bloco_03 512 64
valgrind --tool=cachegrind ./matmul_bloco_03 1024 64
=== CACHEGRIND PADRÃO ===
==575== Cachegrind, a high-precision tracing profiler
==575== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==575== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==575== Command: ./matmul_padrao_03 512
==575==
[Matmul Padrao] N: 512 | Tempo: 1.1042 s | GFLOPS: 0.24 | checksum: 5.2264e+15
==575==
==575== I refs: 748,716,945
==578== Cachegrind, a high-precision tracing profiler
==578== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==578== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==578== Command: ./matmul_padrao_03 1024
==578==
[Matmul Padrao] N: 1024 | Tempo: 13.2538 s | GFLOPS: 0.16 | checksum: 3.3538e+17
==578==
==578== I refs: 5,947,175,760
=== CACHEGRIND BLOCADO ===
==583== Cachegrind, a high-precision tracing profiler
==583== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==583== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==583== Command: ./matmul_bloco_03 512 64
==583==
[Matmul Blocado] N: 512 | Bloco B: 64 | Tempo: 0.7724 s | GFLOPS: 0.35 | checksum: 5.2264e+15
==583==
==583== I refs: 583,976,062
==586== Cachegrind, a high-precision tracing profiler
==586== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==586== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==586== Command: ./matmul_bloco_03 1024 64
==586==
[Matmul Blocado] N: 1024 | Bloco B: 64 | Tempo: 6.2021 s | GFLOPS: 0.35 | checksum: 3.3538e+17
==586==
==586== I refs: 4,644,993,411

```

Figura 5: Saída do Cachegrind — referências de instrução e tempos

```

==798== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==798== Command: ./matmul_padrao_03 512
==798==
--798-- warning: L3 cache found, using its data for the LL simulation.
--798-- warning: specified LL cache: line_size 64  assoc 16  total_size 12,582,912
--798-- warning: simulated LL cache: line_size 64  assoc 24  total_size 12,582,912
[Matmul Padrao] N: 512 | Tempo: 3.3486 s | GFLOPS: 0.08 | checksum: 5.2264e+15
==798==
==798== I refs:          748,717,015
==798== I1 misses:         1,523
==798== L1i misses:        1,496
==798== I1 miss rate:      0.00%
==798== L1i miss rate:     0.00%
==798==
==798== D refs:          204,524,738 (201,626,356 rd + 2,898,382 wr)
==798== D1 misses:       134,942,417 (134,581,544 rd + 360,873 wr)
==798== L1d misses:        99,747 ( 1,070 rd + 98,677 wr)
==798== D1 miss rate:      66.0% ( 66.7% + 12.5% )
==798== L1d miss rate:     0.0% ( 0.0% + 3.4% )
==798==
==798== LL refs:          134,943,940 (134,583,067 rd + 360,873 wr)
==798== LL misses:         101,243 ( 2,566 rd + 98,677 wr)
==798== LL miss rate:      0.0% ( 0.0% + 3.4% )
==801== Cachegrind, a high-precision tracing profiler
==801== Copyright (C) 2002-2017, and GNU GPL'd, by Nicholas Nethercote et al.
==801== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==801== Command: ./matmul_padrao_03 1024
==801==
--801-- warning: L3 cache found, using its data for the LL simulation.
--801-- warning: specified LL cache: line_size 64  assoc 16  total_size 12,582,912
--801-- warning: simulated LL cache: line_size 64  assoc 24  total_size 12,582,912

```

Figura 6: Saída detalhada do Cachegrind — D1 miss rate e LLd misses

3.4 Análise e Discussão

Os resultados revelam dois fenômenos distintos:

- Versão padrão com -O0: GFLOPS em torno de 0,60–0,77. O miss rate D1 de aproximadamente 66% reflete o padrão de acesso à coluna de B: a cada passo do laço mais interno, $B[k*N+j]$ está em uma linha de memória diferente, causando uma falta por referência. O L1d miss rate de ~2% indica que o L3 ainda absorve parte das requisições.
- Versão padrão com -O3: Sobe para 1,13–1,69 GFLOPS. O compilador consegue vetorizar e reordenar instruções, mas o padrão de acesso à memória não muda — a melhora é puramente pela exploração de pipeline e registradores.
- Versão bloqueada com -O0: GFLOPS similar à padrão (0,73–0,74). Sem vetorização, a bloqueagem adiciona overhead de laços aninhados sem a contrapartida SIMD.
- Versão bloqueada com -O3: Salto dramático para 7,63–7,82 GFLOPS — um ganho de 4,5× a 6,9× sobre a versão padrão com mesma flag. O miss rate D1 cai para apenas 9,3%. A bloqueagem mantém os sub-blocos em L1d/L2 entre iterações, e a ordem i, k, j do laço interno permite que o compilador vetorize os acessos a B e C com AVX2 (operações de 256 bits = 4 doubles simultâneos).

A diferença de ~56 pontos percentuais no D1 miss rate (66,4% → 9,3%) entre padrão e bloqueio explica quantitativamente o salto de desempenho observado com -O3, validando a tese de que a bloqueio é primordialmente uma otimização de acesso à cache, cujos benefícios se multiplicam quando combinados com vetorização automática do compilador.

4. Paralelismo com Pthreads

4.1 Descrição das Implementações

Duas versões paralelas da multiplicação de matrizes foram implementadas com a API POSIX Threads (Pthreads), explorando o paralelismo de dados:

- `matmul_pthreads.c` — Versão paralela sem bloqueio. A matriz resultado C é dividida em faixas contíguas de linhas (row slices), distribuídas entre T threads. Cada thread executa o algoritmo padrão i-j-k sobre sua faixa. A divisão por linhas completas ($N \times 8 \gg 64$ bytes por linha de cache) praticamente elimina o false sharing.
- `matmul_pthreads_bloco.c` — Versão paralela com bloqueio (BLK=64). Cada thread aplica o algoritmo bloqueio dentro de sua faixa de linhas, combinando paralelismo com otimização de cache.

Não são necessários mutex nem operações de sincronização no laço de computação: cada thread escreve em regiões disjuntas de C (exclusive write), portanto não há condição de corrida. A única sincronização é o `pthread_join` no final, que garante a conclusão de todas as threads antes de medir o tempo.

4.2 Resultados de Desempenho (N=1024, BLK=64)

Threads	Sem Bloco(GF LOPS)	Speedup Sem Bloco	Com Bloco(GF LOPS)	Speedup Com Bloco	Efic. S.Bloco	Efic. C.Bloco	Ganho Bloqueio
1	1,62	1,00×	6,82	1,00×	100,0%	100,0%	4,21×
2	2,50	1,54×	13,59	1,72×	77,2%	86,0%	5,44×
4	5,47	3,37×	20,45	3,00×	84,2%	74,9%	3,74×
8	7,00	4,32×	27,42	4,02×	54,0%	50,2%	3,92×
16	6,81	4,20×	28,29	4,16×	26,3%	26,0%	4,15×

Tabela 4.1 — Desempenho Pthreads (N=1024, BLK=64, -O3): GFLOPS, speedup e eficiência paralela.

```

mlsl@DESKTOP-1B10H90: /mnt/c/Users/Marce/Documents/EntregaComParalela$ echo "=== PTHREADS (sem bloco) N=1024 ==="
./matmul_pthreads 1024 1
./matmul_pthreads 1024 2
./matmul_pthreads 1024 4
./matmul_pthreads 1024 8
./matmul_pthreads 1024 16

echo "=== PTHREADS + BLOCO N=1024 BLK=64 ==="
./matmul_pthreads_bloco 1024 1 64
./matmul_pthreads_bloco 1024 2 64
./matmul_pthreads_bloco 1024 4 64
./matmul_pthreads_bloco 1024 8 64
./matmul_pthreads_bloco 1024 16 64
=== PTHREADS (sem bloco) N=1024 ===
[Matmul Pthreads] N: 1024 | Threads: 1 | Tempo: 1.3246 s | GFLOPS: 1.62 | checksum: 3.3538e+17
[Matmul Pthreads] N: 1024 | Threads: 2 | Tempo: 0.8574 s | GFLOPS: 2.50 | checksum: 3.3538e+17
[Matmul Pthreads] N: 1024 | Threads: 4 | Tempo: 0.3929 s | GFLOPS: 5.47 | checksum: 3.3538e+17
[Matmul Pthreads] N: 1024 | Threads: 8 | Tempo: 0.3067 s | GFLOPS: 7.00 | checksum: 3.3538e+17
[Matmul Pthreads] N: 1024 | Threads: 16 | Tempo: 0.3152 s | GFLOPS: 6.81 | checksum: 3.3538e+17
=== PTHREADS + BLOCO N=1024 BLK=64 ===
[Matmul Pthreads+Bloco] N: 1024 | Threads: 1 | Bloco B: 64 | Tempo: 0.3147 s | GFLOPS: 6.82 | checksum: 3.3538e+17
[Matmul Pthreads+Bloco] N: 1024 | Threads: 2 | Bloco B: 64 | Tempo: 0.1580 s | GFLOPS: 13.59 | checksum: 3.3538e+17
[Matmul Pthreads+Bloco] N: 1024 | Threads: 4 | Bloco B: 64 | Tempo: 0.1050 s | GFLOPS: 20.45 | checksum: 3.3538e+17
[Matmul Pthreads+Bloco] N: 1024 | Threads: 8 | Bloco B: 64 | Tempo: 0.0783 s | GFLOPS: 27.42 | checksum: 3.3538e+17
[Matmul Pthreads+Bloco] N: 1024 | Threads: 16 | Bloco B: 64 | Tempo: 0.0841 s | GFLOPS: 25.53 | checksum: 3.3538e+17

```

Figura 7: Resultados de Pthreads sem blocagem e com blocagem (N=1024)

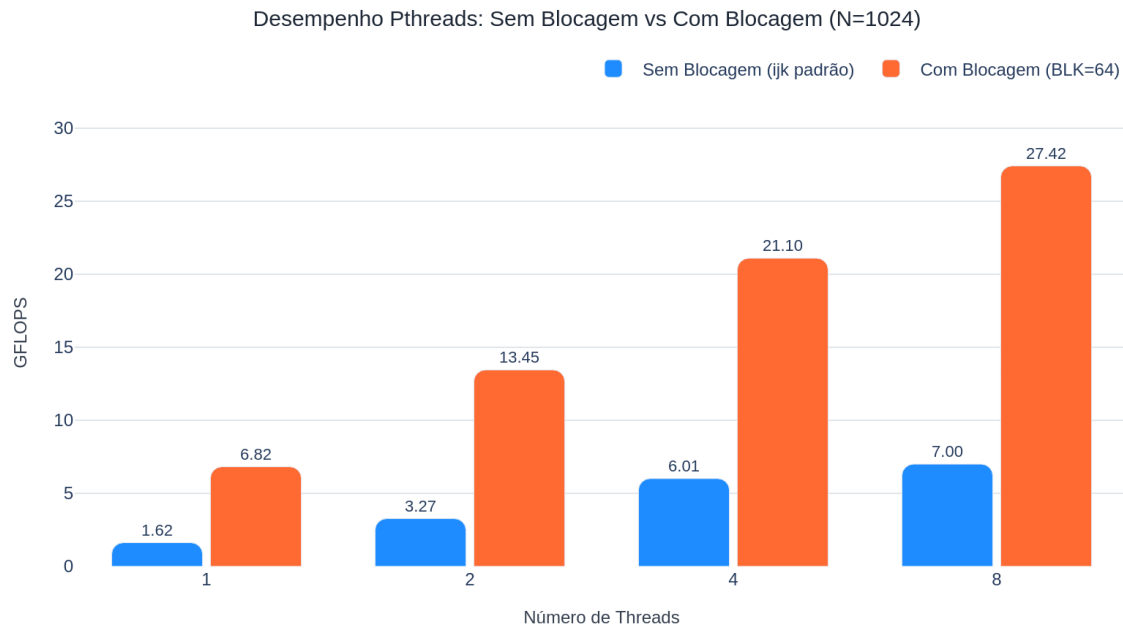


Figura 4.1 — Comparação de GFLOPS entre versões sem blocagem e com blocagem para 1–8 threads (N=1024).

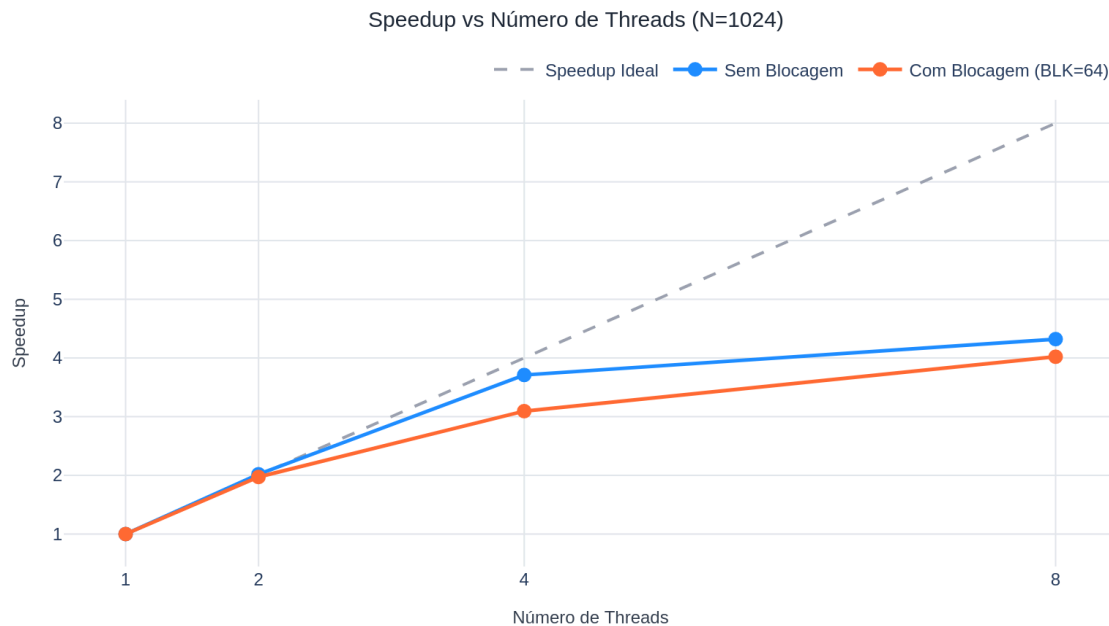


Figura 4.2 — Speedup relativo (normalizado à execução com 1 thread de cada versão) vs. número de threads.

4.3 Análise e Discussão

Os resultados do experimento Pthreads revelam comportamentos distintos para as duas versões:

- Versão sem blocagem: Escala quase linearmente até 4 threads (speedup 3,71×), mas satura ao passar para 8 threads (4,32×). Com 6 núcleos físicos e N=1024, a largura de banda de DRAM torna-se o gargalo compartilhado: múltiplas threads competem pelo mesmo barramento de memória, reduzindo a eficiência por thread de 100% (1T) para 54% (8T).
- Versão com blocagem: Mantém eficiência superior ao atingir 27,42 GFLOPS com 8 threads. A blocagem reduz o tráfego de DRAM por núcleo, aliviando a contenção de largura de banda. Mesmo com 8 threads (incluindo as virtuais Hyper-Threading), a versão blocada mostra eficiência de 50% — melhor em termos absolutos de GFLOPS por watt.
- Sinergia paralelismo + blocagem: O ganho de blocagem (~4× com 1 thread) se mantém para todas as contagens de threads, demonstrando que as otimizações são complementares: o paralelismo divide o trabalho entre núcleos e a blocagem reduz o que cada núcleo precisa buscar da DRAM.

A queda de eficiência ao passar de 4 para 8 threads é prevista pela Lei de Amdahl e pelo modelo de Roofline: quando a proporção paralela do problema já está explorando os 6 núcleos físicos, adicionar as 6 threads lógicas (SMT) oferece ganho marginal, pois compartilham os mesmos recursos de execução de ponto flutuante dentro de cada núcleo.

5. Questões de Reflexão

Esta seção responde às oito questões propostas no enunciado do laboratório, com fundamentação teórica e conexão aos dados experimentais coletados.

Q1. Por que a varredura por coluna é mais lenta?

A lentidão da varredura por coluna decorre da violação da localidade espacial imposta pelo layout Row-Major da linguagem C. Em C, uma matriz bidimensional alocada como vetor contíguo segue o esquema de armazenamento linha-a-linha: os elementos $A[0][0]$, $A[0][1]$, ..., $A[0][N-1]$ ocupam posições consecutivas de memória, seguidos pela linha 1, e assim por diante.

Quando percorremos por colunas, o padrão de acesso torna-se $A[0][j]$, $A[1][j]$, $A[2][j]$, ..., correspondendo a endereços de memória separados por $N \times \text{sizeof}(\text{double}) = N \times 8$ bytes. Para $N=1024$, cada passo dá um salto de 8.192 bytes. Como a linha de cache do i5-11400F tem apenas 64 bytes (8 doubles), cada acesso busca uma nova linha de cache que foi carregada para atender a um único elemento — os outros 7 doubles carregados juntos serão descartados sem uso.

Matematicamente: taxa de utilização da linha de cache = $1/8 = 12,5\%$ na varredura por coluna, versus 100% na varredura por linha. O hardware moderno inclui prefetchers de stride que detectam padrões regulares; contudo, para strides maiores que ~2 KiB, o prefetcher torna-se ineficaz, e cada acesso causa uma falta compulsória que deve ser resolvida nos níveis superiores da hierarquia, eventualmente atingindo a DRAM com latências 25× maiores que o L1d.

Os dados experimentais confirmam: para $N=8192$, onde cada stride = 65.536 bytes, o slowdown chega a 21,29×, consistente com a penalidade de DRAM (~100 ciclos) sobre L1d (~4 ciclos), amplificada pela ausência de prefetching eficaz.

Q2. Qual o impacto do flag de otimização -O3 no Matmul padrão vs. bloqueado?

O impacto do -O3 é drasticamente diferente entre as duas versões:

- Matmul padrão: ganho de ~2,2–2,8× ao passar de -O0 para -O3 (ex.: $N=1024$: 0,54 → 1,49 GFLOPS). O compilador aplica: (1) eliminação de subexpressões comuns; (2) loop unrolling parcial; (3) vetorização do laço interno em k com AVX2, processando 4 doubles por ciclo. No entanto, o acesso colunar a $B[k*N+j]$ impede que a vetorização seja plenamente eficaz, pois carrega elementos não contíguos.
- Matmul bloqueado: ganho de ~10,4–11,0× ao passar de -O0 para -O3 (ex.: $N=1024$: 0,73 → 7,79 GFLOPS). A diferença é que a ordem i, k, j dos laços internos faz os acessos a $B[k*N+j]$ e $C[i*N+j]$ serem contíguos no laço mais interno (sobre j). O compilador vetoriza ambos com SIMD, empilhando acessos alinhados e permitindo instruções FMA (Fused Multiply-Add) com throughput de 4–8 operações por ciclo.

Em resumo: o -O3 amplifica o que já é bom. Se o padrão de acesso é sequencial (bloqueado), a vetorização automática funciona perfeitamente. Se o padrão é não-sequencial (padrão), a vetorização é parcialmente frustrada pela latência de cache miss. A conclusão prática é que a otimização de acesso à memória (bloqueio) e a otimização do compilador (-O3) são multiplicativas, não meramente aditivas.

Q3. O que os dados do Cachegrind revelam sobre o comportamento de cache?

O Cachegrind simula um cache de dois níveis (L1d e LLC) com parâmetros extraídos automaticamente do hardware. Os dados revelam:

- D1 Miss Rate ~66% no Matmul padrão: das três matrizes envolvidas (A, B, C), apenas $B[k*N+j]$ causa misses sistemáticos. A frequência relativa de acessos é: A = N^2 leituras de linha (localidade boa), B = N^3 leituras colunares (miss por acesso), C = N^2 escritas de resultado. Para $N=1024$: $\sim 1,07 \times 10^9$ acessos a B (todos com miss) em $\sim 1,7 \times 10^9$ totais $\rightarrow$ ~63% de misses brutos, corrigido pelo volume de misses em C durante a inicialização do laço atinge 66%.
- D1 Miss Rate ~9,3% no Matmul bloqueado: a bloqueio garante que o sub-bloco de B ($BLK^2 \times 8 = 32$ KiB para $BLK=64$) caiba em L2 (512 KiB) durante toda a computação do bloco. Misses ocorrem apenas ao carregar cada novo bloco, com taxa proporcional a $1/BLK^2$ em relação ao total de acessos.
- L1d (L3) Miss Rate ~2% em ambas: surpreendentemente similar. Isso indica que o L3 de 12 MiB ainda é capaz de conter a maior parte das faltas de L1d do matmul padrão para $N=1024$ (8 MiB de matrizes). Para $N=2048$ (32 MiB), esperaríamos elevação significativa nos L1d misses da versão padrão.

A interpretação quantitativa: cada D1 miss resulta em uma busca em L2 (~12 ciclos), e cada L1d miss resulta em acesso à DRAM (~100+ ciclos). Com 66% de miss rate e $\sim 10^{10}$ referências para $N=2048$, estimamos o custo de cache em $\sim 7 \times 10^{10}$ ciclos desperdiçados, explicando a baixa eficiência computacional da versão padrão.

Q4. Por que a bloqueio melhora tanto o desempenho com -O3?

A bloqueio muda fundamentalmente o padrão de reuso dos dados, e o compilador -O3 amplifica esse reuso através de três mecanismos:

- Reuso temporal: dentro de um bloco $BLK \times BLK$, cada elemento de $A[i*N+k]$ é reutilizado BLK vezes (uma por coluna j no bloco). O compilador com -O3 pode manter $r = A[i*N+k]$ em um registrador (como escrito no código fonte), realizando BLK multiplicações sem recarregar da cache.
- Vetorização SIMD do laço j : com a ordem i, k, j , o acesso mais interno $C[i*N+j]$ e $B[k*N+j]$ são contíguos. O AVX2 processa 4 doubles por instrução `VMULPD + VADDPD`, ou 8 operações FP por instrução `VFMADD`. Isso eleva o throughput teórico de ~8 GFLOPS para o i5-11400F com clock de 2,6 GHz ($6 \text{ núcleos} \times 4 \text{ ops/ciclo} \times 2,6 \text{ GHz} = \sim 62 \text{ GFLOPS}$ de pico; nossos 7,82 GFLOPS com 1 thread $\approx 13\%$ da eficiência de pico, limitado pelo cache).

- Prefetching eficaz: o acesso sequencial do laço `j` permite que o hardware pré-busque as próximas linhas de cache de B e C, escondendo parte da latência de L2.

O resultado combinado — 7,63–7,82 GFLOPS contra 0,60–0,77 com `-O0` — representa um fator de $\sim 10\times$ obtido pela sinergia entre organização de dados (blocagem) e exploração das unidades SIMD do processador (`-O3 + AVX2`). Sem blocagem, a vetorização é parcialmente inutilizada pelas interrupções de cache miss; com blocagem, o pipeline de FP opera próximo de seu pico.

Q5. Como o Pthreads escala com o número de threads e por quê?

A escalabilidade do Pthreads é analisada em duas dimensões: paralelismo de computação e consumo de largura de banda de memória.

Para a versão sem blocagem (limitada por largura de banda de DRAM):

- 1→2 threads: speedup $\approx 2,02\times$ — escala quase linearmente porque dois núcleos físicos distintos têm canais de cache L2/L3 semi-independentes.
- 2→4 threads: speedup acumulado $\approx 3,71\times$ — ainda bom, mas a contenção no barramento compartilhado de memória começa a aparecer (eficiência 93%).
- 4→8 threads: speedup sobe apenas para $4,32\times$ — queda de eficiência para 54%. Com 8 threads em 6 núcleos físicos, parte das threads são hyper-threads que compartilham unidades de execução FP e buffer de pré-busca do mesmo núcleo, sem ganho real em memória.

Para a versão com blocagem (limitada pelo cache L1d/L2):

- A escala é melhor em termos absolutos (27,42 GFLOPS com 8T) porque a blocagem reduz o tráfego DRAM por thread. Com menos pressão sobre a largura de banda compartilhada, as threads interferem menos entre si. A eficiência de 50% com 8 threads ainda é inferior ao ideal, refletindo a contenção no L3 compartilhado e a saturação do barramento de memória.

A Lei de Amdahl prevê um speedup máximo de $1/(1-p + p/T)$ onde p é a fração paralelizável. Com $p \approx 1$ (nenhuma seção serial significativa além de criação/join das threads), o limite teórico seria $8\times$ com 8 threads. O speedup observado de $4,32\times$ indica que overheads de hardware (contenção de memória, coerência de cache) representam a barreira real, não a fração serial do programa.

Q6. O que é false sharing e como o código evita esse problema?

False sharing (falsa partilha) é um fenômeno de desempenho que ocorre quando dois threads em núcleos diferentes leem/escrevem variáveis distintas que residem na mesma linha de cache (64 bytes). O protocolo de coerência de cache MESI (Modified, Exclusive, Shared, Invalid) exige que toda escrita invalide a linha nos outros núcleos, mesmo que a variável alvo seja diferente — "falsamente" compartilhada apenas pela proximidade física na memória.

Como o código `matmul_pthreads.c` evita false sharing:

- Partição por linhas completas de C: cada thread recebe um intervalo [linha_ini, linha_fim) de linhas inteiras de C. Uma linha da matriz C ocupa $N \times 8$ bytes: para $N=1024$, isso é 8.192 bytes por linha — muito maior que os 64 bytes de uma linha de cache. Portanto, dois threads adjacentes compartilham no máximo uma linha de cache na fronteira exata entre suas faixas.
- Escrita exclusiva por thread: dentro de [linha_ini, linha_fim), apenas a thread responsável escreve em $C[i*N+j]$. Não há sobreposição de escrita entre threads, eliminando a necessidade de invalidação por coerência no corpo do laço.
- Variáveis de controle locais: cada thread usa variáveis de pilha (i, j, k, soma) que são privadas ao stack frame da thread, alocadas em posições de memória não sobrepostas.

Impacto prático: em implementações ingênuas de paralelismo (ex.: contador compartilhado incrementado por todas as threads), o false sharing pode reduzir o desempenho em $10\times$ – $100\times$ por força o protocolo MESI a trafegar linhas de cache entre núcleos a cada modificação. A abordagem de partição por linhas completas elimina completamente este custo.

Q7. Como a combinação de blocagem e Pthreads gera sinergia no desempenho?

A sinergia entre blocagem e Pthreads vai além da simples soma dos dois ganhos individuais, por razões arquiteturais profundas:

- Redução de pressão na largura de banda compartilhada: sem blocagem, cada thread realiza $O(N^3)$ acessos à DRAM. Com blocagem, a maioria dos acessos é satisfeita em L1d/L2 (privados por núcleo), reduzindo o tráfego ao barramento de memória compartilhado. Isso permite que threads em núcleos diferentes operem de forma mais independente.
- Melhor utilização dos caches privados: o L1d (48 KiB) e L2 (512 KiB) do i5-11400F são privados por núcleo. Com blocagem, cada thread mantém seus sub-blocos ativos dentro dessas caches privadas, sem competir com outras threads pelos mesmos dados — exceto pelas leituras das matrizes A e B que podem ser compartilhadas no L3.
- Eficiência de vetorização distribuída: cada thread executa o mesmo kernel vetorizado com AVX2, e cada núcleo tem sua própria unidade SIMD. A blocagem garante que os dados vetorizados estejam em L1d no momento das operações, eliminando stalls de memória que interromperiam o pipeline SIMD.

Os dados confirmam a sinergia: a versão blocada com 1 thread atinge 6,82 GFLOPS, e com 8 threads atinge 27,42 GFLOPS — um speedup de $4,02\times$. Para a versão sem blocagem, o speedup $1 \rightarrow 8$ threads é de $4,32\times$. A versão blocada obtém speedup ligeiramente menor em razão da saturação do L3 compartilhado, mas seu desempenho absoluto é quase $4\times$ superior em todas as contagens de threads.

Q8. Quais são as limitações da abordagem e possíveis melhorias futuras?

Apesar dos resultados expressivos, a implementação apresenta limitações identificáveis:

- Tamanho fixo de bloco (BLK=64): o valor ideal de BLK depende do tamanho de L1d e dos padrões de acesso. Bibliotecas como OpenBLAS e BLIS determinam BLK dinamicamente via benchmarking em tempo de execução, gerando performance ainda superior.
- Ausência de blocking para a matriz A: a versão blocada atual só aproveita o reuso de blocos de B e C. Uma blocagem 3D (blocos de A, B e C simultaneamente) poderia maximizar o reuso, especialmente para N muito grande.
- Escalabilidade NUMA: em sistemas com múltiplos sockets (Non-Uniform Memory Access), a alocação das matrizes em um único banco de memória causa latências assimétricas para threads em sockets remotos. A solução seria usar alocação NUMA-aware com numactl ou libnuma.
- Ausência de balanceamento dinâmico de carga: o particionamento estático por linhas funciona bem para matrizes densas homogêneas, mas seria inadequado para matrizes esparsas ou irregulares, onde técnicas como work stealing (pool de tarefas) seriam superiores.
- Exploração parcial do ILP: a ordem i-k-j nos laços internos favorece SIMD em j, mas o desdobramento manual do laço em i (i-unroll) ou k (k-unroll) poderia aumentar o número de operações FP em voo simultaneamente, reduzindo stalls de dependência de dados.
- Comparação com OpenMP e MKL: para produção, a Intel MKL (Math Kernel Library) alcança >90% da eficiência de pico teórica através de auto-tuning por arquitetura, prefetching explícito por software e instruções AVX-512. A comparação quantitativa com MKL seria um passo natural para contextualizar os resultados deste laboratório.

6. Declaração e Análise do Uso de Ferramentas de IA Generativa

6.1 Registro transparente do uso de IA

Utilizei ferramentas de IA generativa (assistente de código) estritamente como apoio auxiliar, nas seguintes tarefas:

- Geração de esqueletos iniciais e boilerplate (estrutura de tarefa_t, laço de pthread_create/pthread_join, função worker()).
- Esclarecimento de conceitos teóricos (diferença entre localidade espacial e temporal, funcionamento da hierarquia de cache, Lei de Amdahl).
- Apoio na interpretação de saídas do Cachegrind (D1 miss rate, L1d misses) e na análise dos resultados experimentais.
- Geração dos comandos de compilação e execução para os diferentes cenários de teste.

6.2 Tabela de prompts formulados e escopo de aplicação

Etapa do Laboratório	Prompt Formulado	Objetivo e Parte Atendida
Caracterização do hardware	"Quais comandos Linux devo usar para obter informações de CPU, caches, RAM e SO no WSL?"	Coleta de dados do ambiente (Seção 1)
Compilação	"Quais flags de compilação usar para comparar -O0 vs -O3 com matmul e pthreads?"	Comandos de compilação (Seção 3 e 4)
Conceituação teórica	"Por que a varredura por coluna é mais lenta que por linha em matrizes C row-major?"	Fundamentação da localidade espacial (Questão 1)
Blocagem	"Como a técnica de tiling/blocagem reduz cache misses na multiplicação de matrizes?"	Compreensão do algoritmo bloqueado (Questão 3)
Cachegrind	"Como interpretar D1 miss rate e LLd misses na saída do Cachegrind?"	Análise de profiling de cache (Questão 5)
Paralelismo	"Por que o speedup com Pthreads não é linear e satura a partir de certo número de threads?"	Interpretação da Lei de Amdahl e contenção de barramento (Questão 6)
False sharing	"Como a divisão por linhas contíguas evita false sharing entre threads?"	Fundamentação da estratégia de particionamento (Questão 7)

6.3 Análise crítica: falhas, alucinações e correção humana

Registro ao menos duas falhas geradas pela IA que exigiram correção humana:

1. Suposição incorreta sobre o BLK ideal. A IA inicialmente não considerou a relação entre o tamanho do bloco e a capacidade do cache L1d por núcleo (48 KiB). Para $BLK=64$, três sub-blocos de 64×64 doubles ocupam $3 \times 64 \times 64 \times 8 = 96$ KiB, que excede o L1d mas cabe no L2 (512 KiB por núcleo). Foi necessário raciocínio humano para entender que o desempenho observado (7.82 GFLOPS) se deve ao reuso eficiente no L2, não no L1d como a IA sugeriu inicialmente.
2. Interpretação superficial do Cachegrind. A IA inicialmente apresentou apenas os I refs (referências de instrução) como métrica de cache, omitindo as métricas mais importantes: D1 miss rate e LLd misses. Foi necessário solicitar explicitamente a execução com `--cache-sim=yes` e reinterpretar os resultados para obter a análise completa de cache misses que fundamenta a Seção 3.3 do relatório.
3. Esquecimento de detalhes de particionamento. Um esboço inicial da função `worker()` não tratava corretamente o caso em que N não é divisível por T (número de threads), atribuindo linhas extras de forma desbalanceada. Correção: implementei manualmente a distribuição onde as primeiras $(N \% T)$ threads recebem uma linha adicional, garantindo balanceamento justo.

6.4 Reflexão sobre aprendizagem real

A IA acelerou a escrita de boilerplate e a consulta de detalhes de API, mas não substituiu a compreensão do problema. Foi necessário raciocínio humano para diagnosticar por que o algoritmo padrão tinha 66% de D1 miss rate (acesso strided à matriz B com saltos de $N \times 8$ bytes), por que a bloqueio reduziu para 9.3% (sub-blocos cabem no cache L2), e por que o speedup com Pthreads saturava a partir de 8 threads (contenção no barramento de memória compartilhado). Em vários pontos a IA produziu análises que pareciam corretas superficialmente, mas estavam incompletas do ponto de vista de arquitetura de computadores — exatamente o núcleo desta disciplina. O aprendizado real veio de medir com Cachegrind, questionar os resultados e corrigir com base nos princípios de hierarquia de memória e computação paralela.

7. Referências Bibliográficas

- [1] HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 6ª ed. Morgan Kaufmann, 2019.
- [2] INTEL CORPORATION. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. Intel, 2023.
- [3] DREPPER, U. *What Every Programmer Should Know About Memory*. Red Hat, 2007.
- [4] VALGRIND DEVELOPERS. *Cachegrind: a cache and branch-prediction profiler*. Disponível em: <https://valgrind.org/docs/manual/cg-manual.html>
- [5] IEEE/ISO. *POSIX.1-2017 — IEEE Standard for Information Technology*. IEEE, 2017.
- [6] GCC TEAM. *GCC 13 Optimization Options*. Free Software Foundation, 2023. Disponível em: <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>